Network design practice 03

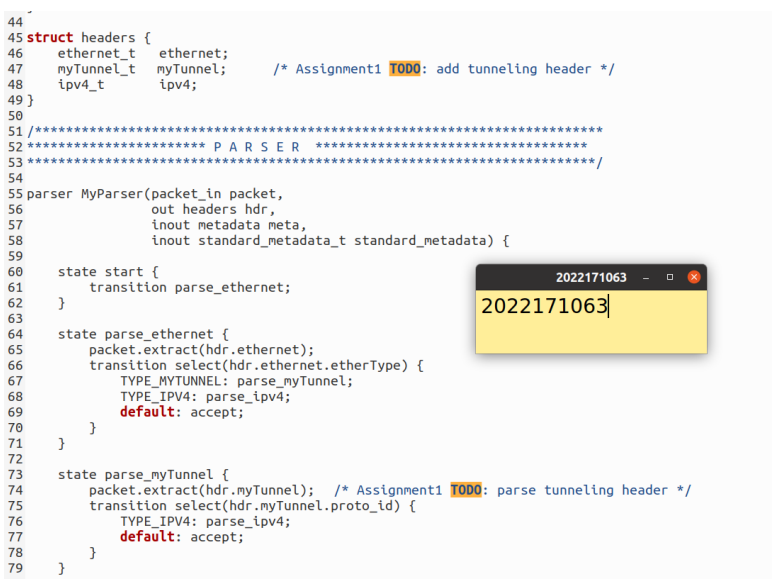
김경민

March 31, 2026

1 Overview

This practice simulates network tunneling.

2 Assignment 1

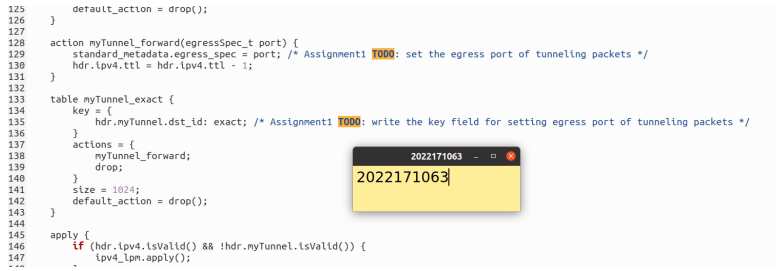


```
struct headers {
    ethernet_t   ethernet;
    myTunnel_t   myTunnel;      /* Assignment1 TODO: add tunneling header */
    ipv4_t       ipv4;
}

/*************************************************************************
********************** P A R S E R  ******************************
*************************************************************************/

parser MyParser(packet_in packet,
                out headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        transition parse_ethernet;
    }

    state parse_ethernet {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            TYPE_MYTUNNEL: parse_myTunnel;
            TYPE_IPV4: parse_ipv4;
            default: accept;
        }
    }

    state parse_myTunnel {
        packet.extract(hdr.myTunnel);   /* Assignment1 TODO: parse tunneling header */
        transition select(hdr.myTunnel.proto_id) {
            TYPE_IPV4: parse_ipv4;
            default: accept;
        }
    }
```


Figure 1: Adding tunneling header

Added tunneling header in structure headers referencing how ethernet header is added. I passed the argument hdr.myTunnel for packet to extract. See

ethernet header is extracted in similar manner.



```
        default_action = drop();
    }

    action myTunnel_forward(egressSpec_t port) {
        standard_metadata.egress_spec = port; /* Assignment1 TODO: set the egress port of tunneling packets */
        hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
    }

    table myTunnel_exact {
        key = {
            hdr.myTunnel.dst_id: exact; /* Assignment1 TODO: write the key field for setting egress port of tunneling packets */
        }
        actions = {
            myTunnel_forward;
            drop;
        }
        size = 1024;
        default_action = drop();
    }

    apply {
        if (hdr.ipv4.isValid() && !hdr.myTunnel.isValid()) {
            ipv4_lpm.apply();
```


In the lower block, the destination address is set to dst_id. Note that it is different from dstAddr of ethernet/ipv4 headers. The code checks the forwarding table with hdr.myTunnel.dst_id as key, finds port, and hand over the port number to forwarding action.

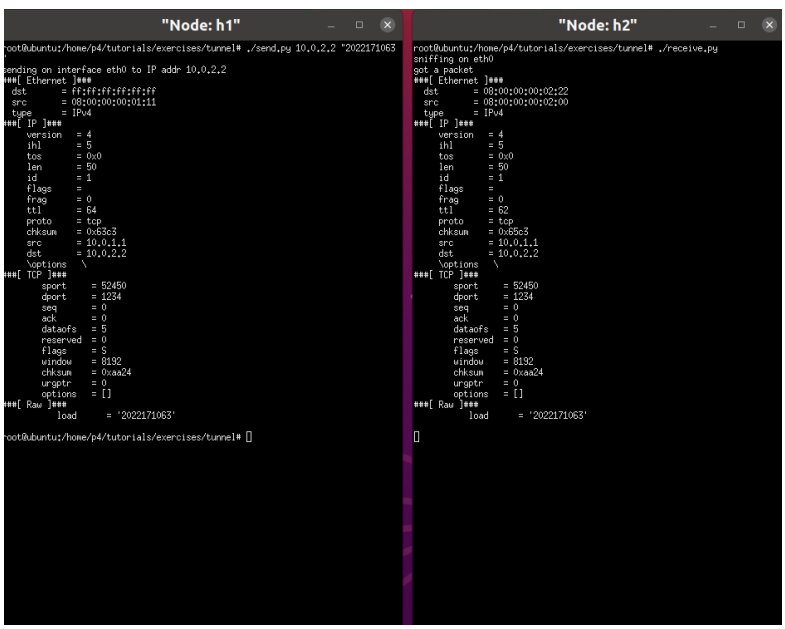


Figure 2: ipv4 packets

The packet has been sent without setting dst_id argument. Checking the Ethernet type on h2 side(receiving side) we get ipv4, which means the packet was sent in ipv4.

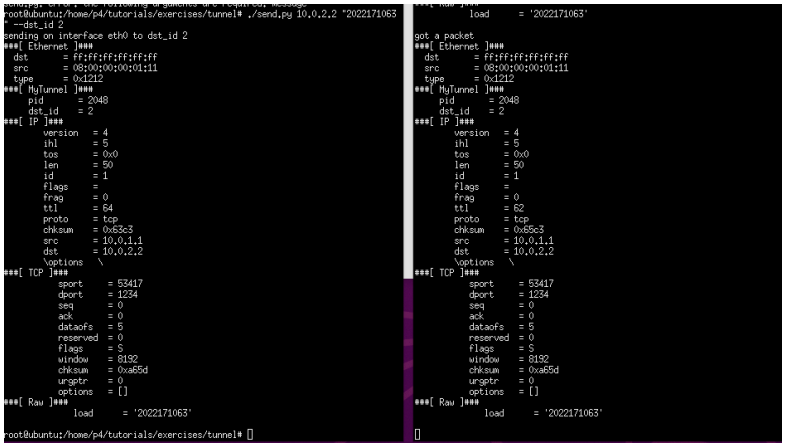


Figure 3: tunneling packets

We passed argument dst_id as 2, and now it shows the packet is sent on tunneling method. The ethernet type shows 0x1212 which is TYPE_MYTUNNEL id specified in the top of tunnel.p4 file.

3 Assignment 2

```
        },
        {
          "table": "MyIngress.myTunnel_exact",
          "match": {
            "hdr.myTunnel.dst_id": [
              3
        ]
          },
          "action_name": "MyIngress.myTunnel_forward",
          "action_params": {
            "port": 2
          }
        },
        {
          "table": "MyIngress.myTunnel_exact",
          "match": {
            "hdr.myTunnel.dst_id": [
             2
            ]
          },
          "action_name": "MyIngress.myTunnel_forward",
          "action_params": {
            "port": 3
          }
        }
```

1063

Figure 4: Modifying port

s1-runtime.json file specifies which port to send packet when s1 switch receives one. Now, I switched port number 2 and 3 to make packets bound for h2 actually head to s3. This way we can easily modify tunneling path from s1 s2 to s1 s3 s2.

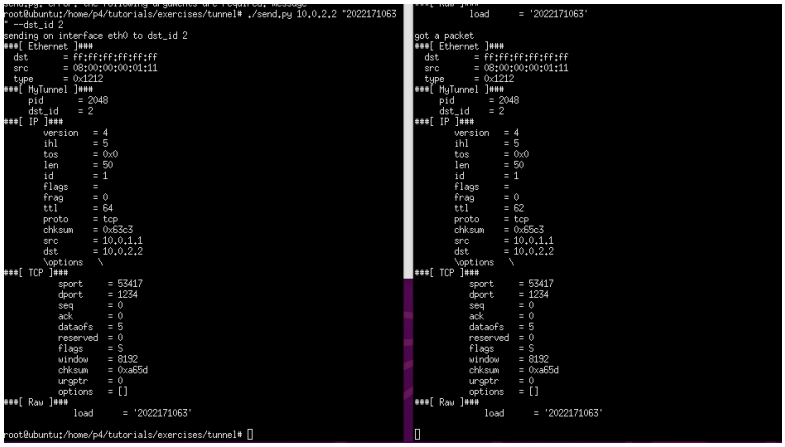


Figure 5: Before

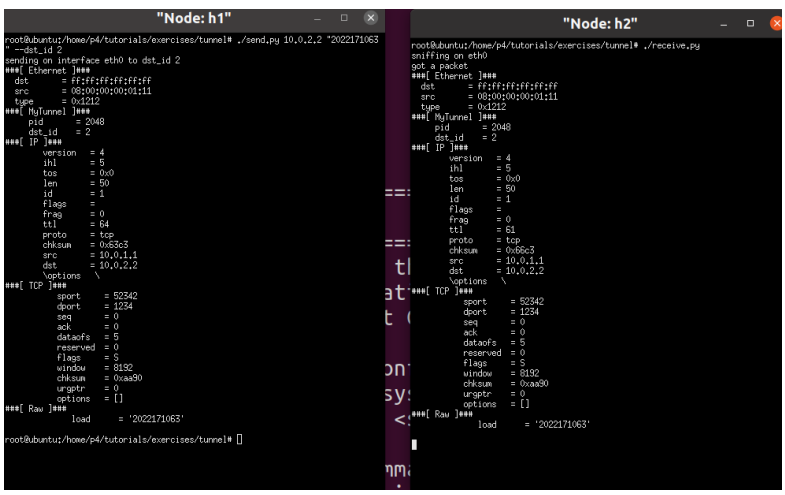


Figure 6: After

Let us compare ttl before and after modifying the code. TTL shows how many "hops" the packet did before reaching its destination. Before the modification ttl dropped from 64 to 62(2 hops), and after it dropped from 64 to 61(3 hops). This confirms that we successfully rerouted the packet path.

4 Assignment 3

The header consists of Ethernet/Tunneling/IP header and comes in front of the payload. tunnel.p4 contains the size of headers on top of the lines, which gives the overall header size 304 bit (38 byte). Combined with payload it gvies

$$38 + 64 = 102 \tag{1}$$

Since the tunneling header is 4 byte, the packet overhead ratio of the tunneling header would be 4 divided by 102.

$$4/102 = 3.92\% \tag{2}$$

The overhead ratio of the header is

$$38/102 = 37.25\% \tag{3}$$

Tunneling method secure remote working scenarios, and is a useful tool to virtualize network system. It also allows legacy or proprietary traffic to slip into modern infrastructure by encapsulating protocols. However, the overhead is larger to enable virtual environment, and at times heavily used tunnel can create bottlenecks, increasing latency.